

Hardware and Software Fundamentals of Intelligent Computing Systems

AI System Fundamentals & Heterogeneous Computing

Guojie Luo

2026/8/4

Outline

- **AI Computing and Heterogeneous Hardware**
- Heterogeneous Programming Fundamentals
- Triton Programming Basics
- Lab: AI Operators and GEMM Tuning

Token-Hungry Generative AI Applications

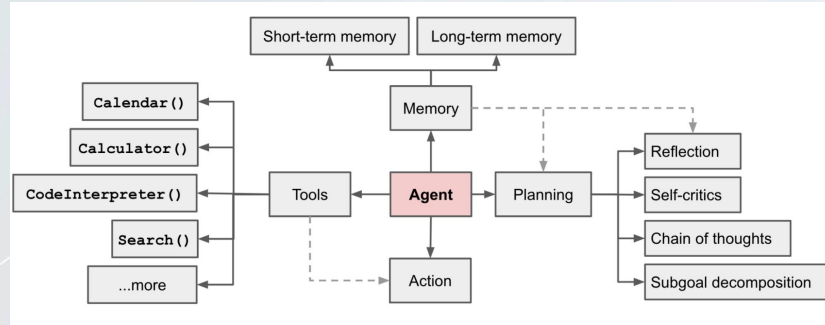


BAAI
智源研究院

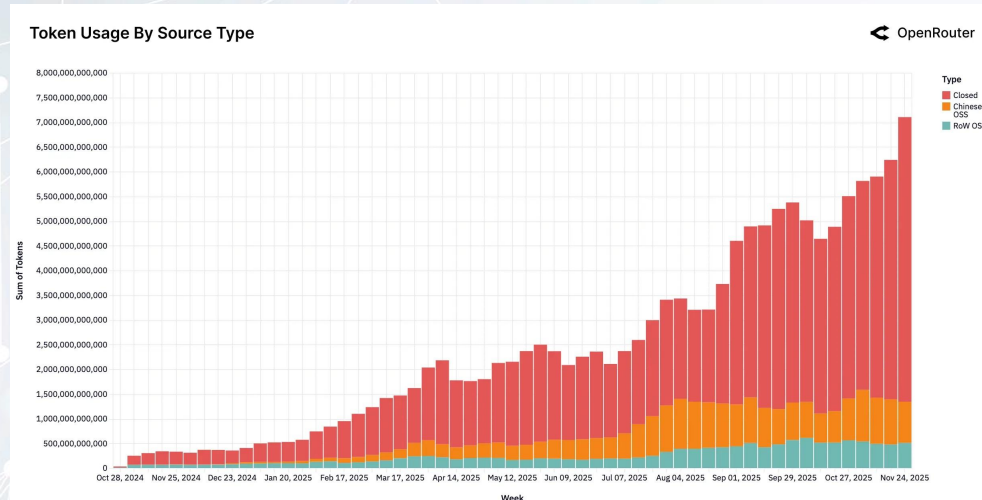


北京大学
PEKING UNIVERSITY

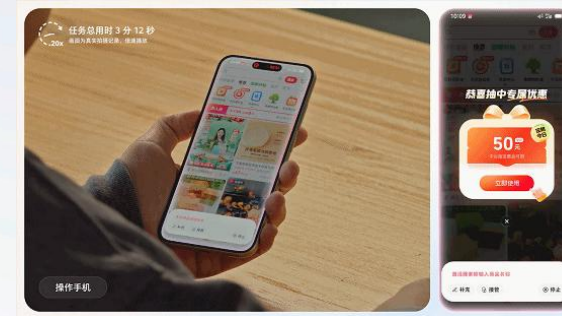
- LLM-powered autonomous agents



- Token consumption trends



- AI mobile phone and AI PC



- Video generation



The Hardware Demands of AI/ML

- Model Parameters
 - The exponential growth.
- Computational Demand
 - Outpacing the Moore's Law.
 - Comparison between compute cost and U.S GDP by OpenAI.
- Communication Scale
 - The new critical bottleneck.

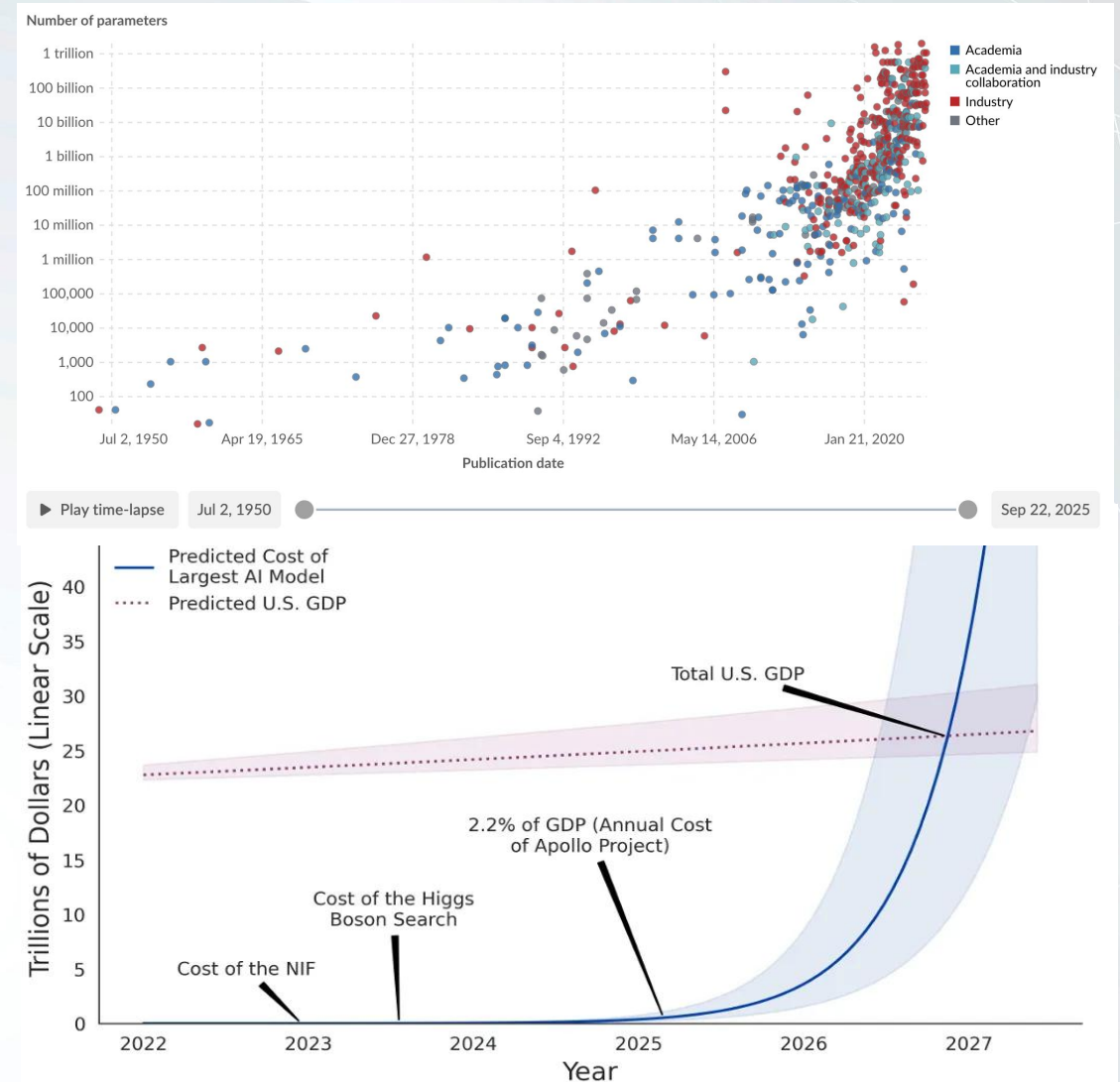
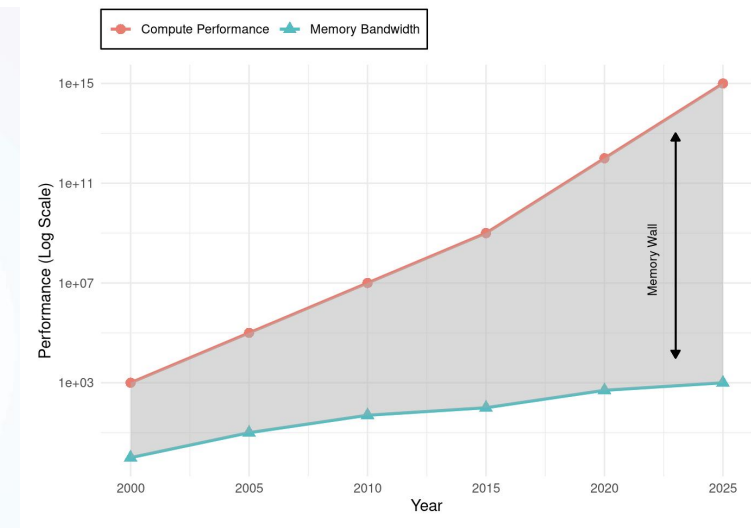
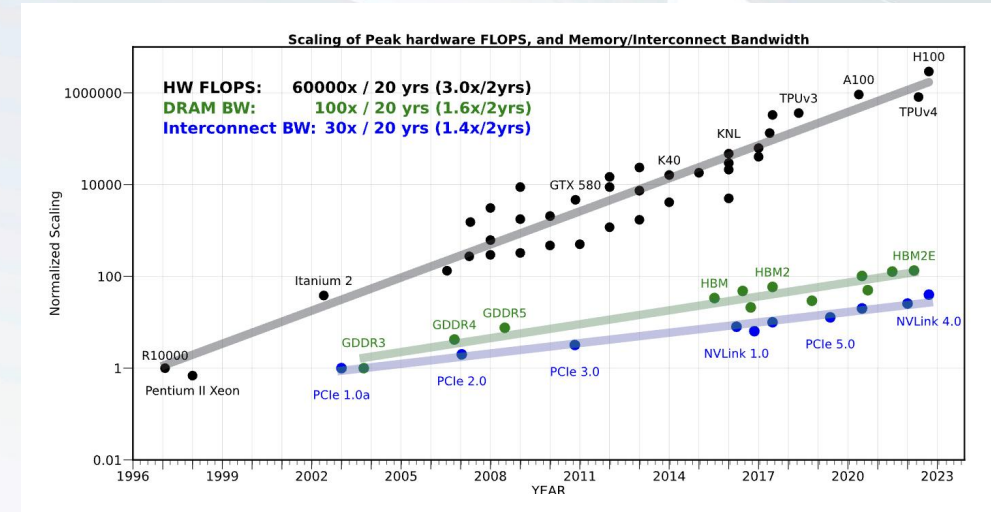


image: <https://medium.com/geekculture/business-analysis-ai-computational-cost-67a136957c95>

<https://ourworldindata.org/grapher/artificial-intelligence-parameter-count>

Why General-Purpose CPUs Failed AI

- The CPU's Legacy:
 - For decades, the Central Processing Unit (CPU) was the universal engine of computing, designed for sequential, general-purpose tasks.
- The Two Crises:
 - Von Neumann Bottleneck
 - End of Performance Scaling



Why General-Purpose CPUs Failed AI

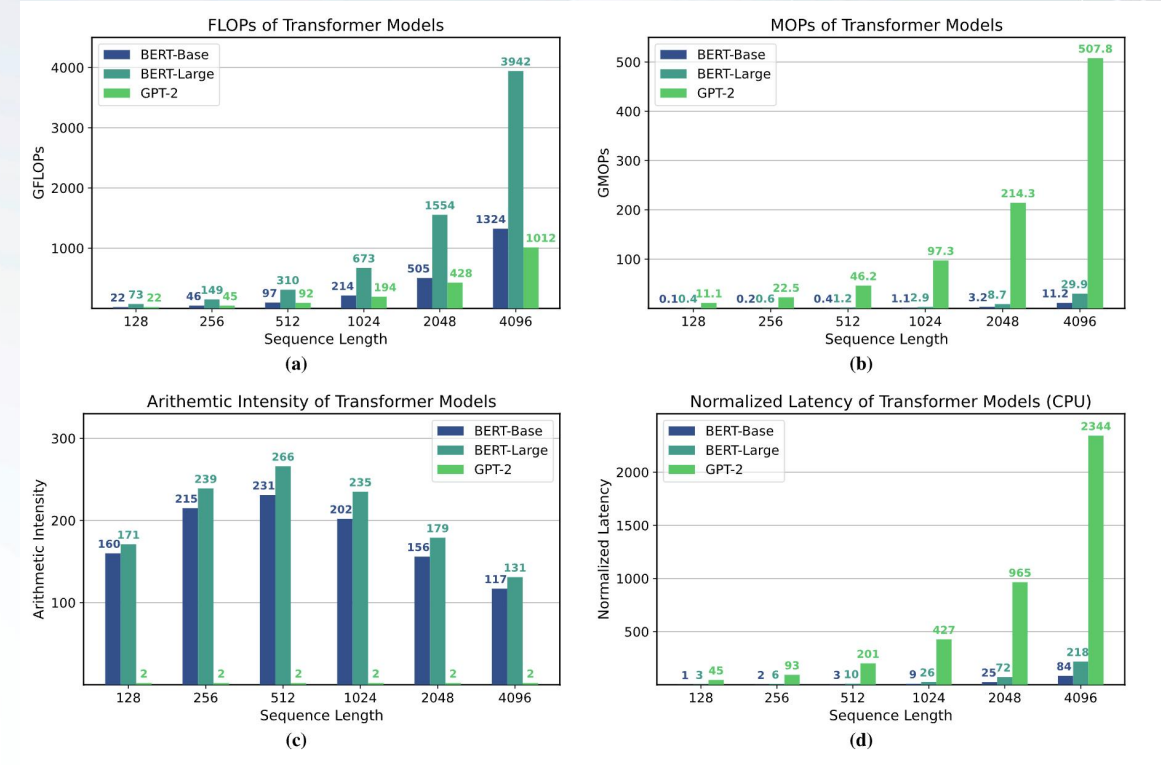


BAAI
智源研究院



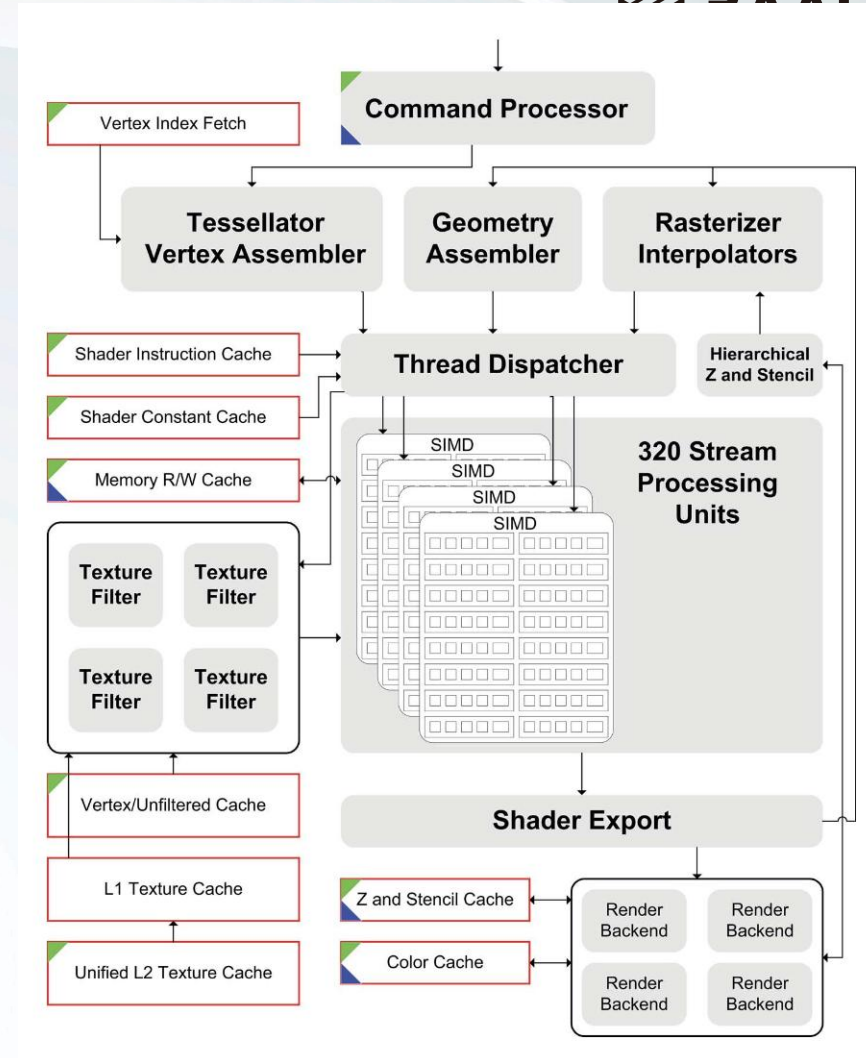
北京大学
PEKING UNIVERSITY

- The Mismatch:
 - CPUs' sequential design is fundamentally inefficient for the massive parallelism inherent in AI workloads like matrix multiplication.
 - *CPU's achieve only 5-10% utilization on neural network workloads, delivering approximately 100 GFLOPS¹⁰ while consuming hundreds of watts*.*
- Conclusion:
 - AI's exponential demands exposed the limits of the general-purpose model, forcing a shift to specialized hardware.



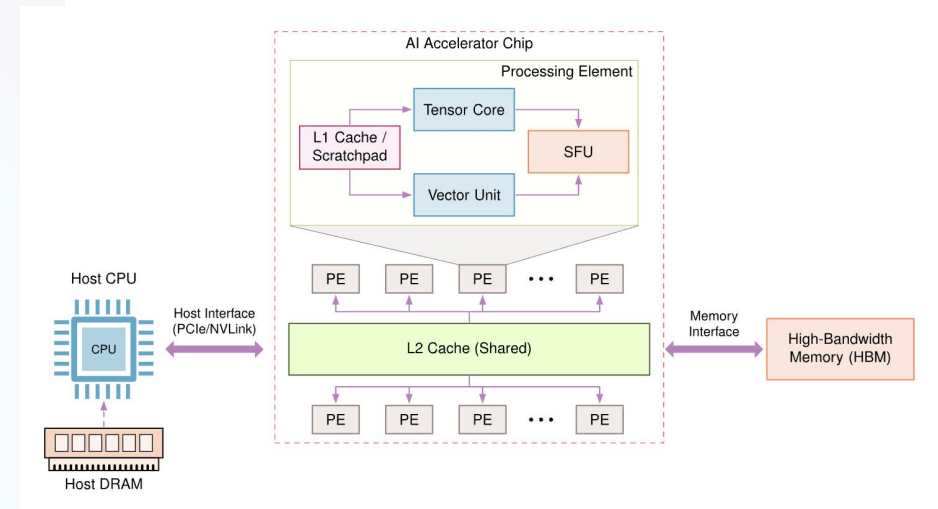
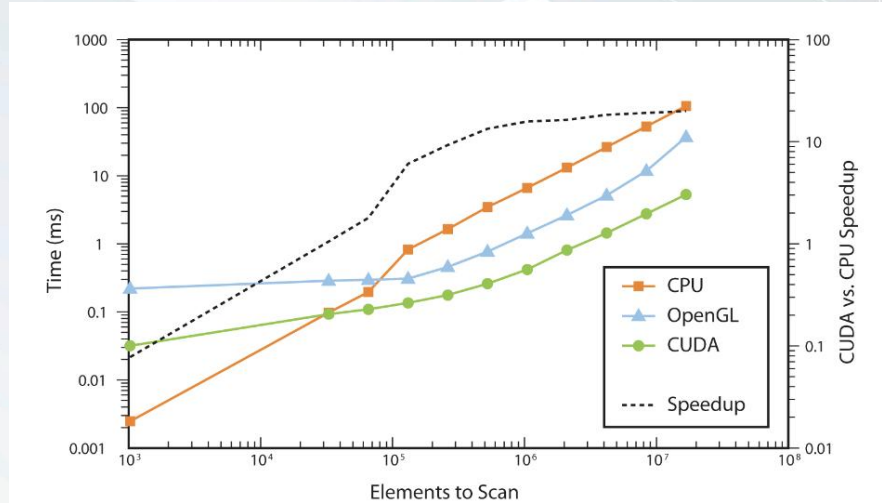
The Rise of the GPU

- An Unlikely Hero: The Graphics Processing Unit (GPU) was born in the late 1990s for video game graphics, featuring thousands of simple cores for parallel tasks.
- Architectural Resonance: The core math of graphics (matrix and vector operations) was identical to the math of deep learning, making GPUs a perfect, albeit accidental, fit.



The Rise of the GPU

- The Software Catalyst: CUDA: NVIDIA's CUDA platform (2007) unlocked the GPU for general-purpose computing, making its power accessible to AI researchers without deep graphics expertise.
- The Result: Training times for AI models plummeted from weeks to hours, establishing the GPU as the de facto standard for AI training.



AI Hardware Spectrum: Key Examples



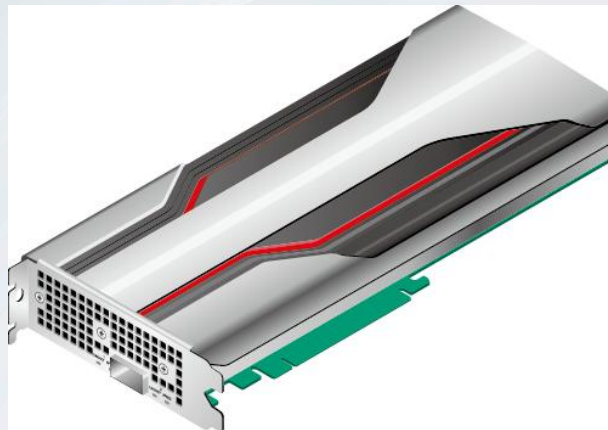
BAII
智源研究院



北京大学
PEKING UNIVERSITY

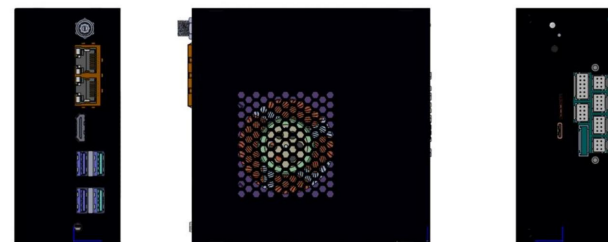
Cloud ML

Huawei Ascend
Cambricon
MLU370
etc.



Edge ML

Iluvatar
TY1200



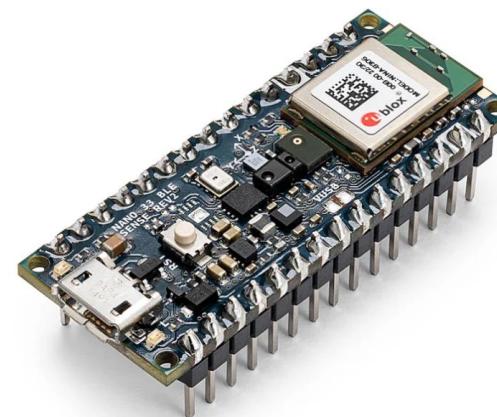
Mobile ML

Kirin 9030 Pro



Tiny ML

Arduino Nano 33
BLE Sense



Compute Architectures Evolve



- GPU series (+ Tensor cores)
- TPU series
- Ascend series
- CPU series: Intel (+ matrix extension); AMD (+ NPU)
- compute-in-memory
- wafer-scale chip
- Supernode and Hypercomputer
- ...

Different Processors Optimize Different Trade-offs

CPU	GPU	NPU
Optimized for low latency and complex control	Optimized for highly parallel throughput	Optimized for common AI dataflows
A small number of powerful cores	Many parallel execution lanes	Matrix and vector engines with on-chip buffers
Large caches and branch prediction	SIMT execution and high-bandwidth memory	High efficiency for supported operators and data types
Best suited for orchestration, preprocessing and control-heavy workloads	Best suited for tensor operations and programmable kernels	Usually driven by compiler, graph and framework toolchains

- Modern AI systems combine CPUs and accelerators rather than relying on a single processor type.

Iluvatar BI-V150: A General-Purpose GPU for AI Computing



1. Product Positioning

- General-Purpose GPU
 - Developed by Iluvatar CoreX for AI training, inference and general-purpose computing
 - Available as an air-cooled PCIe accelerator card
 - Designed for deployment in multi-card and multi-node computing systems



Iluvatar BI-V150: A General-Purpose GPU for AI Computing



2. Hardware Architecture

Specification	BI-V150
GPU architecture	ivcore11
Process technology	7 nm
Device memory	64 GB HBM2e
Host interface	PCIe 4.0 ×16
Peak board power	350 W

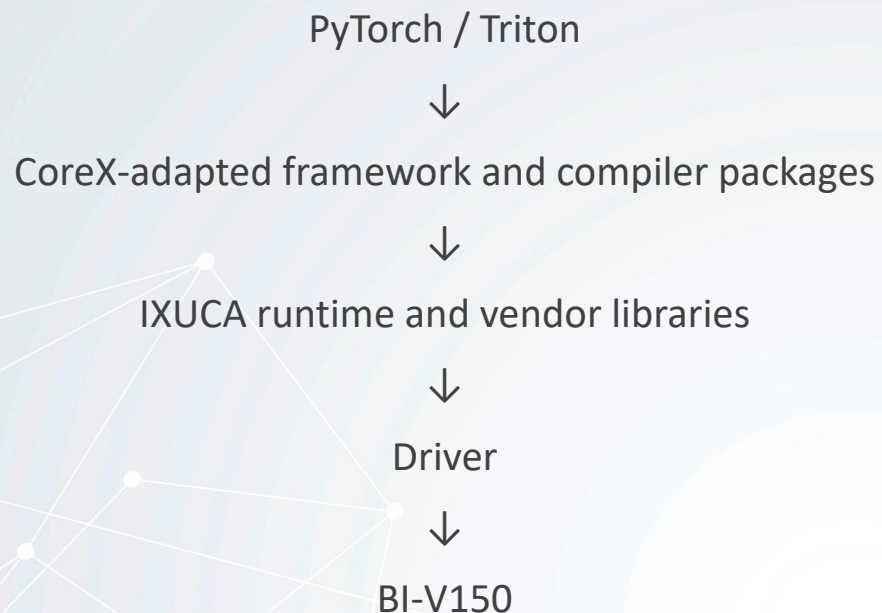


Iluvatar BI-V150: A General-Purpose GPU for AI Computing



3. Execution and Programming Model

- CUDA-Compatible, but Architecturally Different



- Grid -> Thread Block -> Warp (**64 threads**) -> Thread



Outline



BAAI
智源研究院



北京大学
PEKING UNIVERSITY

- AI Computing and Heterogeneous Hardware
- **Heterogeneous Programming Fundamentals**
- Triton Programming Basics
- Lab: AI Operators and GEMM Tuning

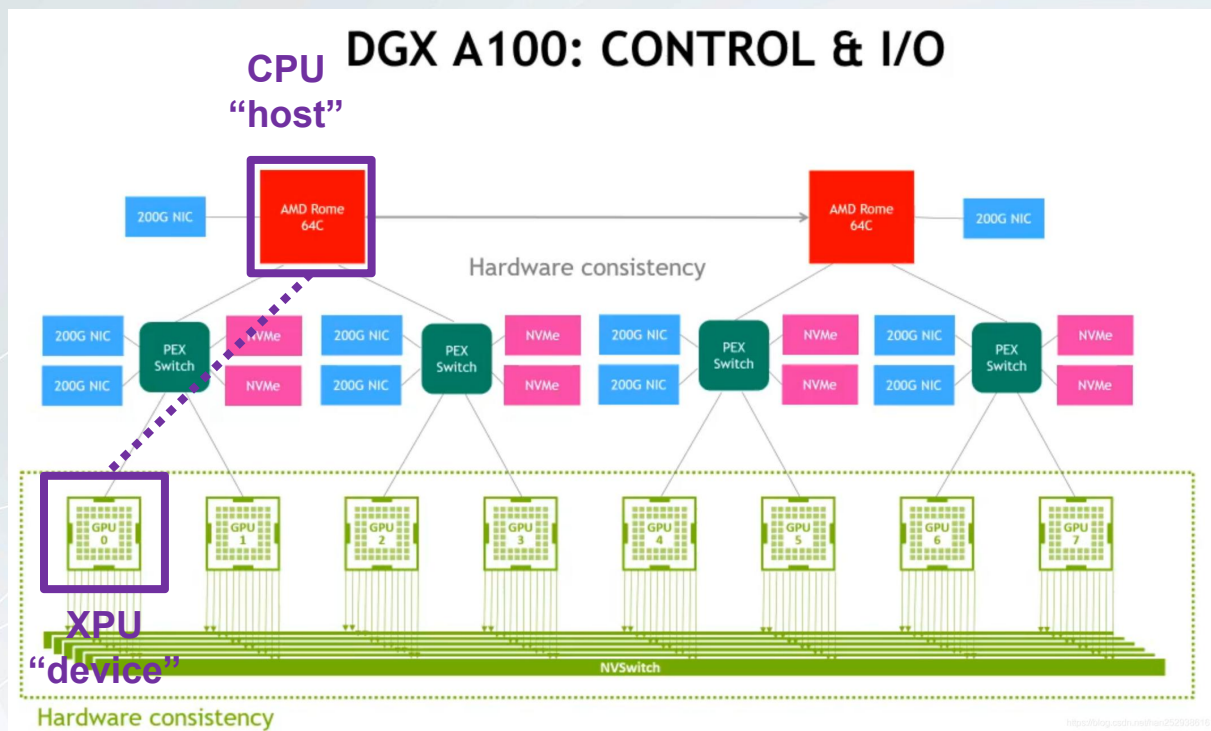
Questions on Heterogeneous Programming



- How to compile the “kernel” code?
- How do CPU and GPU interact?
- How to express parallelism?
- How to express synchronization?
- How to manage and access data?
- How is kernel code executed on GPU?

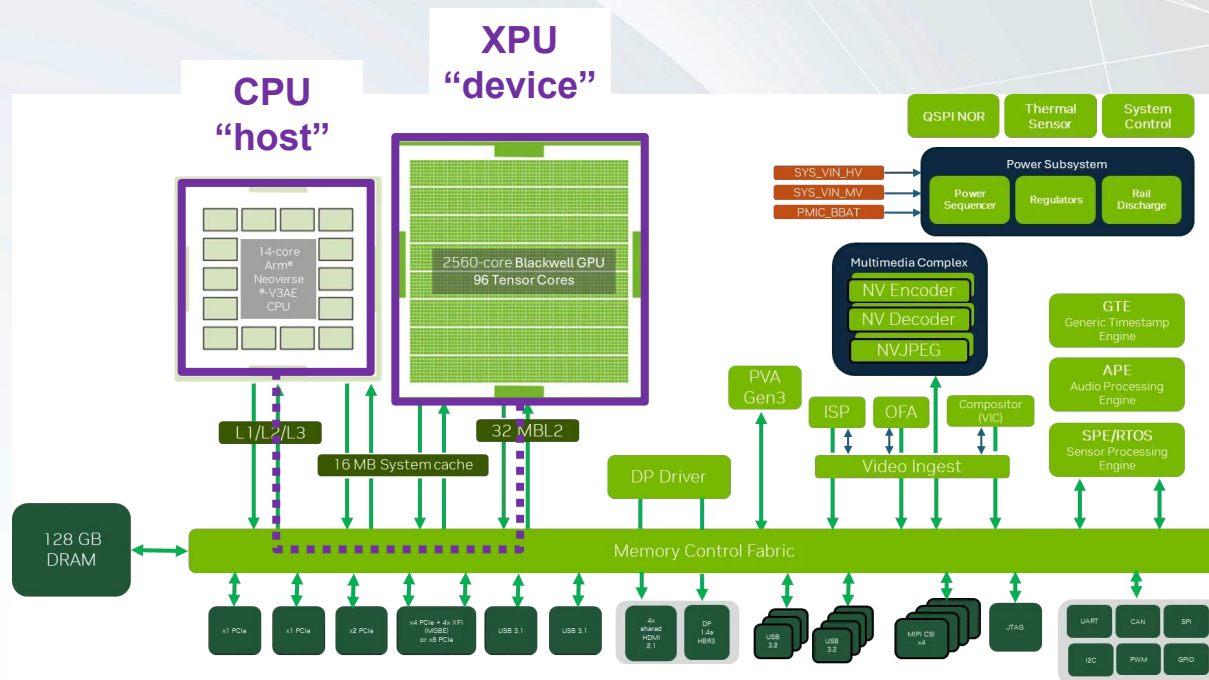
Heterogeneous Integration of CPU and XPU

- Inter-chip (board-level) integration



- protocols: PCIe, NVLink, etc.

- Intra-chip integration



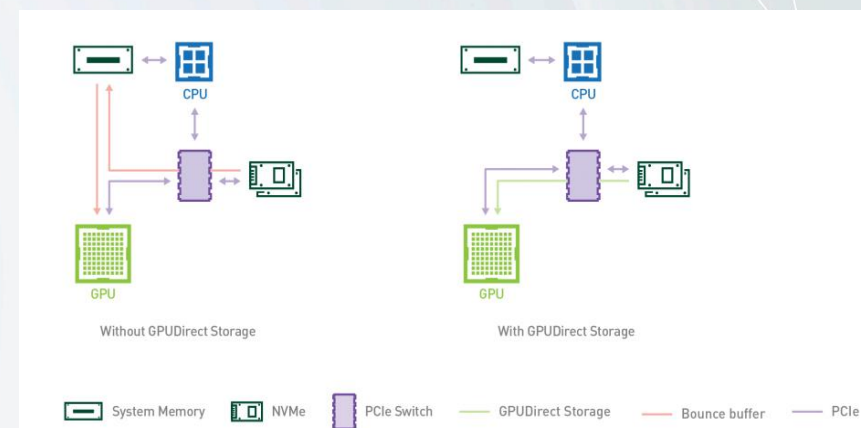
- protocols: one-way or two-way coherency

PCIe Role in AI: GPU-Host Connection

- GPU connect to most Host with PCIe
 - Host->GPU: Commands (CUDA kernels, OpenCL bin)
 - Training data, Model weights...
 - GPU->Host: GPU status (kernel completion, CUDA events)
 - Computation results (gradients, inference output)...
- Traditional Data Transfer
 - 1st Copy: NVMe SSD -> host Memory [with DMA]
 - 2nd Copy: host Memory -> GPU Memory
- Nvidia GPUDirect Storage (from 2020.06)
 - Only 1 Copy: NVMe SSD -> GPU Memory



GPUs to mainboard with PCIe



Hosts Submit Work; Streams Order Kernels

- KERNEL → KERNEL

- Kernel A writes an intermediate tensor
- Same stream: ordered dependency
- Kernel B reads the tensor

- Events express dependencies across streams.

CUDA
Streams · Events · Device / Unified Memory

- HOST → KERNEL

- Allocate device tensors
 - Enqueue copies and kernel launches
 - Continue without waiting for completion
 - Synchronize before reading results
- A launch is usually asynchronous.

CANN
Device · Context · Stream · aclrtMemcpyAsync

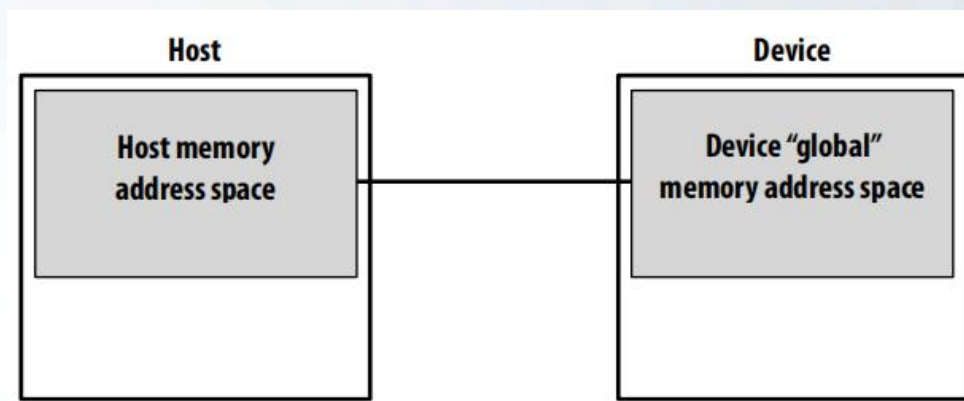
How to Manage and Access Data?

- CUDA memory model
 - Distinct host and device address spaces (when not using unified memory)
 - Memory allocation in GPU through `cudaMalloc()`
 - Move data between address spaces through `cudaMemcpy()`

```
int main()
{
    // executed on CPU
    int A[32] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ... 31};
    int B[32] = {9, 8, 7, 6, 5, 4, 3, 2, 1, 0, ... 2};
    int C[32] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ... 0};

    int *g_A, *g_B, *g_C;
    cudaMalloc((void**)&g_A, 32 * sizeof(int));
    cudaMalloc((void**)&g_B, 32 * sizeof(int));
    cudaMalloc((void**)&g_C, 32 * sizeof(int));
    cudaMemcpy(g_A, A, 32 * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(g_B, B, 32 * sizeof(int), cudaMemcpyHostToDevice);

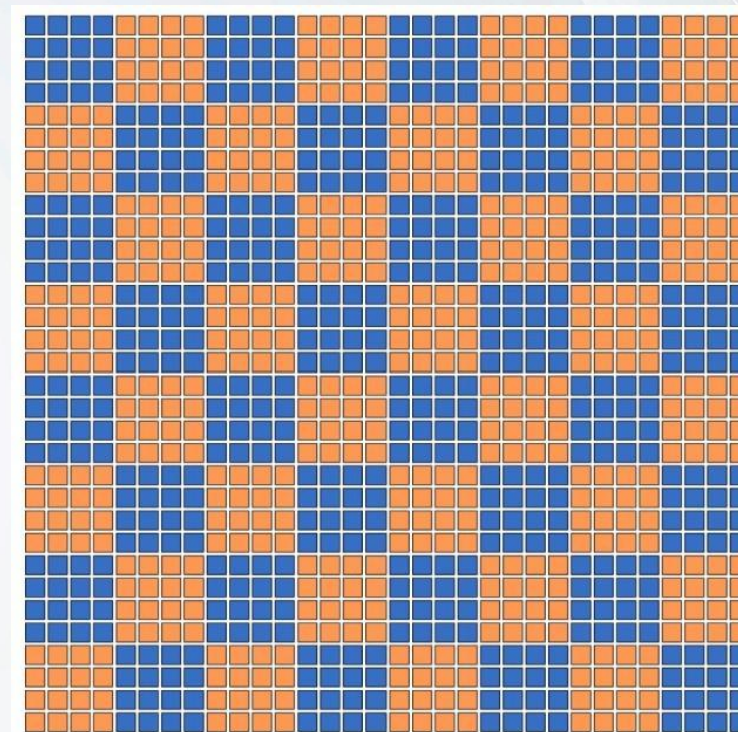
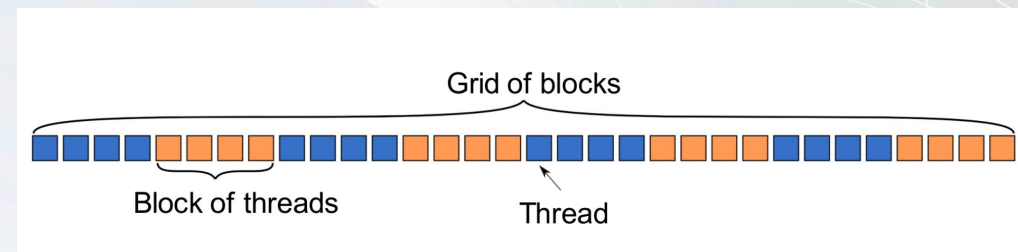
    int numBlocks = 1;
    int numThreads = 32;
    vadd <<<numBlock,numThreads>>>(g_A,g_B,g_C);
    cudaMemcpy(C, g_C, 32 * sizeof(int), cudaMemcpyDeviceToHost);
}
```



How to Express Parallelism?



- 1D Grid
- Threads in grid have access to:
 - Their respective block: `blockIdx.x`
 - Their respective thread ID in a block: `threadIdx.x`
 - Their block's dimension: `blockDim.x`
 - The number of blocks in the grid: `gridDim.x`
- 2D Grid
- Threads in grid have access to:
 - Their respective block IDs: `blockIdx.x`, `blockIdx.y`
 - Their respective thread IDs in a block: `threadIdx.x`, `threadIdx.y`



How to Express the Work of a Thread

- A device function that will be launched from the host program is called a kernel & is declared with the `__global__` attribute
- Kernels should be declared void
- All pointers passed to kernels must point to memory on the device
- Threads are grouped into warps and scheduled concurrently when resources permit.
- Each thread uses its unique thread and block IDs to compute a global ID
- There could be more than N threads in the grid
- Kernels are launched from the host:

A simple embarrassingly parallel loop:

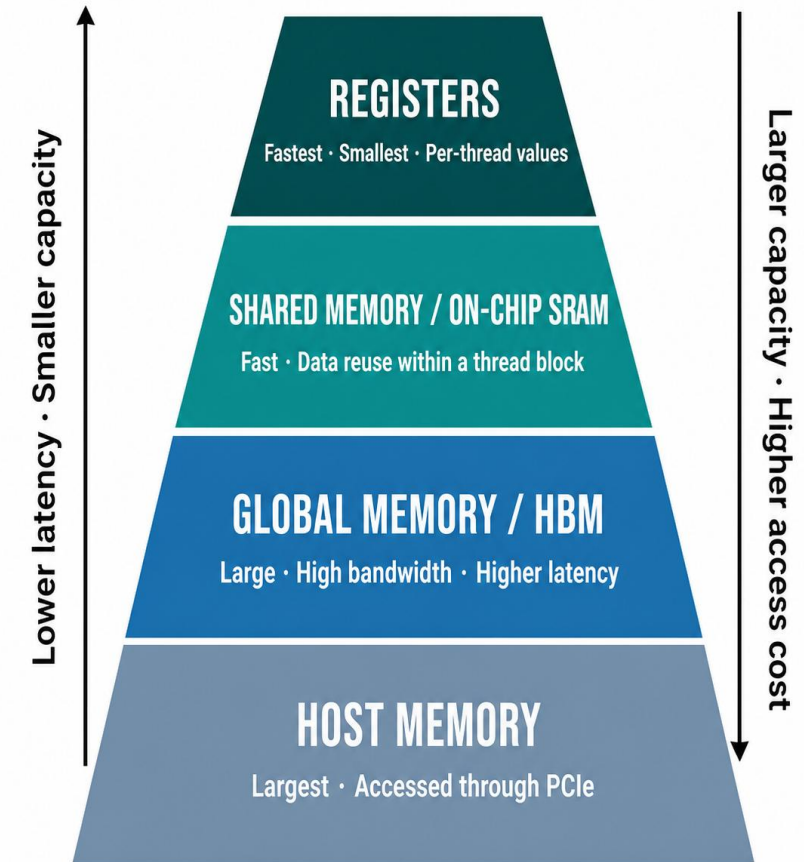
```
for (int i= 0; i<N; i++)  
  
    h_a[i] *= 2.0;
```

Can be translated into a GPU kernel:

```
__global__ void myKernel(int N, double *d_a) {  
    int i = threadIdx.x + blockIdx.x * blockDim.x;  
    if (i<N) d_a[i] *= 2.0;  
}
```

GPU Memory Hierarchy: Capacity, Latency, and Data Reuse

- Why Does the Hierarchy Matter?
 - Global memory provides capacity, but unnecessary traffic is expensive.
 - Registers hold thread-local intermediate values.
 - Shared memory enables threads in a block to reuse data.
 - Coalesced accesses improve effective global-memory bandwidth.
 - Tiling loads data once and reuses it for multiple computations.



Lab connection: GEMM tiles reuse blocks of A and B across many multiply-accumulate operations. Larger tiles may increase reuse, but also consume more registers and on-chip resources.

Three Ways to Express a Kernel

CUDA SIMT	CUDA Tile	Ascend C
Unit: individual thread	Unit: compile-time tile	Unit: operator tile
Grid, block, threadIdx	Grid of tile blocks; compiler maps threads	Host tiling + AI Core blocks
Explicit indexing, shared memory and synchronization	Load, compute and store whole tiles	CopyIn → Compute → CopyOut
NVIDIA CUDA target	cuTile Python / CUDA Tile C++	Ascend AI Core / CANN

- Triton is also tile-oriented, but it uses its own DSL, compiler, and target backend.

Outline

- AI Computing and Heterogeneous Hardware
- Heterogeneous Programming Fundamentals
- **Triton Programming Basics**
- Lab: AI Operators and GEMM Tuning

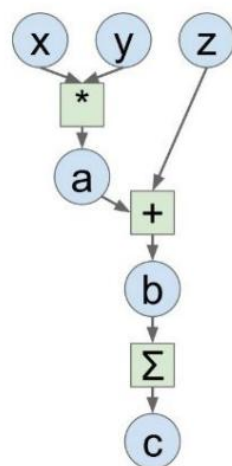
PyTorch Provides the High-Level Operator Interface

- PyTorch: The (most) popular open-source ML library.
 - Tensor definition; GPU-enabled computation; Automatic Differentiation; Compiler Support.



Other ML Frameworks

Computation Graph



Numpy

```
import numpy as np
np.random.seed(0)

N, D = 3, 4

x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

a = x * y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * y
grad_y = grad_a * x
```

Tensorflow

```
import numpy as np
np.random.seed(0)
import tensorflow as tf

N, D = 3, 4

with tf.device('/gpu:0'):
    x = tf.placeholder(tf.float32)
    y = tf.placeholder(tf.float32)
    z = tf.placeholder(tf.float32)

    a = x * y
    b = a + z
    c = tf.reduce_sum(b)

grad_x, grad_y, grad_z = tf.gradients(c, [x, y, z])

with tf.Session() as sess:
    values = {
        x: np.random.randn(N, D),
        y: np.random.randn(N, D),
        z: np.random.randn(N, D),
    }
    out = sess.run([c, grad_x, grad_y, grad_z],
                    feed_dict=values)
    c_val, grad_x_val, grad_y_val, grad_z_val = out
```

PyTorch

```
import torch

N, D = 3, 4

x = torch.rand((N, D), requires_grad=True)
y = torch.rand((N, D), requires_grad=True)
z = torch.rand((N, D), requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()
```

Numpy vs TensorFlow vs PyTorch

Why Triton?



- High-Performance GPU Kernels with Pythonic Ease
- Productivity-Performance Dilemma
 - Frameworks like PyTorch
 - High productivity, but "fusing" custom operators often requires using CUDA C++ to avoid overhead
 - CUDA C++
 - Offers maximum hardware utilization but is notoriously difficult to write, debug, and maintain
 - The Gap
 - Most researchers cannot afford weeks of development time to optimize a single CUDA kernel
- Triton's "Secret Sauce"
 - Block-Level Abstraction
 - Don't manage individual threads; you manage blocks of data.
 - Automatic Parallelization
 - The compiler handles memory coalescing, shared memory allocation, and hardware scheduling.
 - OpenAI Backing
 - Developed specifically to make GPU programming accessible for the next generation of AI models.

Case Study: Vector Addition Kernel

```
import triton
import triton.language as tl

@triton.jit
def add_kernel(
    x_ptr, # Pointer to first input vector
    y_ptr, # Pointer to second input vector
    out_ptr, # Pointer to output vector
    n_elements, # Size of the vector
    BLOCK_SIZE: tl.constexpr, # Compile-time constant
):
    # 1. Identify which "Tile" this program handles
    pid = tl.program_id(axis=0)

    # 2. Calculate memory offsets
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)

    # 3. Create a mask to avoid out-of-bounds access
    mask = offsets < n_elements

    # 4. Load data from DRAM to Registers
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)

    # 5. Perform the operation and Store back to DRAM
    output = x + y
    tl.store(out_ptr + offsets, output, mask=mask)
```

```
BLOCK_SIZE = 256
output = torch.empty_like(x)
grid = (triton.cdiv(n_elements, BLOCK_SIZE),)

add_kernel[grid](
    x, y, output, n_elements,
    BLOCK_SIZE=BLOCK_SIZE,
)
```

host wrapper

Triton Programs Operate on Data Tiles

- Host side
 - `grid` defines the number and shape of program instances
 - `kernel[grid](...)` submits work asynchronously
- Inside each Triton program
 - `tl.program_id` selects the problem tile
 - `tl.arange` creates vector or tile offsets
 - `tl.load` / `tl.store` move masked blocks of data
 - `tl.max`, `tl.sum` and `tl.dot` express collective tile operations
- The compiler handles
 - mapping tile operations to target execution lanes
 - much of the low-level coordination and instruction selection
- You still control grid shape, masks, layout, precision and meta-parameters.
- What Triton Simplifies
 - Maps block/tile operations onto hardware execution lanes.
 - Handles much of vectorization, instruction selection, shared-memory placement and synchronization.
- What Remains on the Host Side
 - PyTorch/runtime owns tensor allocation and memory placement.
 - `kernel[grid](...)` is enqueued asynchronously on the current stream.
 - Kernels exchange data through device tensors.
 - Stream ordering and events express inter-kernel dependencies.

Triton Execution Model

- From Threads to Blocks
 - CUDA Model: Focuses on Threads and Warps. Developers must manually manage synchronization and shared memory.
 - Triton Model: Focuses on Programs and Blocks. You write code that operates on a "tile" (Block) of data as a single unit.
- Key Components
 - The Grid (Coordinate Space) The Grid is a 1D, 2D, or 3D space of execution instances. When you launch a kernel, you define the grid size (e.g., `grid=(1024, )`). Each point in this grid is called a Program.
 - The Program (Instance) A Program is an independent execution instance of your kernel. Programs run in parallel across the GPU's Compute Units. Each program identifies itself using `tl.program_id(axis)`.
 - The Block (Data Unit) A Block is a contiguous chunk of data (a "tile") processed by one Program. Constraint: Block sizes must be powers of two (e.g., 128, 256, 512, 1024) to allow for hardware-level optimizations.

Kernel Anatomy & Pointer Arithmetic

- Defining a Triton Kernel
 - The `@triton.jit` Decorator: Transforms a standard Python function into a GPU-executable kernel.
 - Function Signature:
 - Pointers: Memory addresses for inputs/outputs (e.g., `x_ptr`).
 - Metadata: Size constants or strides (e.g., `n_elements`). `tl.constexpr`: Special arguments that must be known at compile-time (like `BLOCK_SIZE`).
- The Core Logic: Pointer Arithmetic
 - Identify the Program: `pid = tl.program_id(axis=0)` Calculate the Start Offset: `block_start = pid * BLOCK_SIZE`
 - Generate a Range of Offsets: `offsets = block_start + tl.arange(0, BLOCK_SIZE)`
 - Construct Pointers: `pointers = base_ptr + offsets`
- Visualizing the Address Map
 - `tl.arange(0, 4) -> [0, 1, 2, 3]`
 - `base_ptr + [0, 1, 2, 3] -> A vector of 4 memory addresses.`

Memory Operations — Load, Store, and Masking



- The Data Bridge: Global Memory (DRAM) $\leftrightarrow$ SRAM/Registers
 - `tl.load(pointers, mask)`
 - Reads a block of data from the addresses specified by the pointers.
 - Efficiency: Triton automatically ensures "Coalesced Memory Access," meaning the hardware reads data in the most efficient bursts possible.
 - `tl.store(pointers, data, mask)`
 - Writes the computed results from the registers back to Global Memory.
- The "Masking" Essential
 - Because Triton requires Block Sizes to be powers of two (e.g., 512), but your actual data might not be (e.g., 500 elements), you must prevent out-of-bounds memory access.
 - The Logic: Create a boolean mask where `offsets < total_elements`.
 - The Syntax: Python `mask = offsets < n_elements` `data = tl.load(ptr + offsets, mask=mask)`
 - Benefit: If the mask is False, the operation is safely ignored, preventing segmentation faults.

Common Built-in Functions & Reduction



- Element-wise Math
 - Arithmetic: `+`, `-`, `*`, `/`, `**`
 - Math: `tl.exp()`, `tl.log()`, `tl.sqrt()`, `tl.sin()`, `tl.cos()`
 - Comparison: `==`, `!=`, `<`, `>`, `tl.where(condition, x, y)`
- Reduction Operations
 - `tl.sum(data, axis=...)`: Computes the sum along the specified axis.
 - `tl.max(data, axis=...)`: Finds the maximum value.
 - `tl.argmin()` / `tl.argmax()`: Finds the index of the min/max.
- Atomic Operations
 - `tl.atomic_add(pointer, value, mask)`
 - `tl.atomic_max(pointer, value, mask)`

Outline

- AI Computing and Heterogeneous Hardware
- Heterogeneous Programming Fundamentals
- Triton Programming Basics
- **Lab: AI Operators and GEMM Tuning**

Day 1: From Vector Tiles to Reductions and GEMM

Vector Add	Softmax & LayerNorm	Tiled GEMM
1-D grid, offsets and mask	One program per row	One program per output tile
Masked load and store	tl.max / tl.sum reductions	2-D offsets and strides
Protect the tail block	Stable formulas and fp32 statistics	K-loop and tl.dot
Test irregular lengths	Correct padding identities	fp16 I/O; fp32 accumulator

- Correctness tests include exact blocks and boundary-crossing shapes.

GEMM Reuses Tiles Across the K-loop

```
# One program computes one C[BM, BN] tile
pid_m = ...
pid_n = ...
offs_m = pid_m * BM + tl.arange(0, BM)
offs_n = pid_n * BN + tl.arange(0, BN)
offs_k = tl.arange(0, BK)

acc = tl.zeros((BM, BN), dtype=tl.float32)

for k in range(0, tl.cdiv(K, BK)):
    a = tl.load(a_ptrs, mask=(offs_m[:, None] < M)
                & (k * BK + offs_k[None, :] < K), other=0.0)
    b = tl.load(b_ptrs, mask=(k * BK + offs_k[:, None] < K)
                & (offs_n[None, :] < N), other=0.0)
    acc += tl.dot(a, b)
    a_ptrs += BK * stride_ak
    b_ptrs += BK * stride_bk

tl.store(c_ptrs, acc.to(tl.float16), mask=c_mask)

# Three boundaries matter: M, N and K.
```

Day 2: Tune GEMM with Controlled Experiments



1 Calibrate the baseline

Correctness → warmup → synchronize → repeated samples → median

2 Sweep block dimensions

Isolate BM, BN and BK; include symmetric and asymmetric tiles

Record compile failure, wrong result, correct-but-slow and fast

3 Autotune a filtered search space

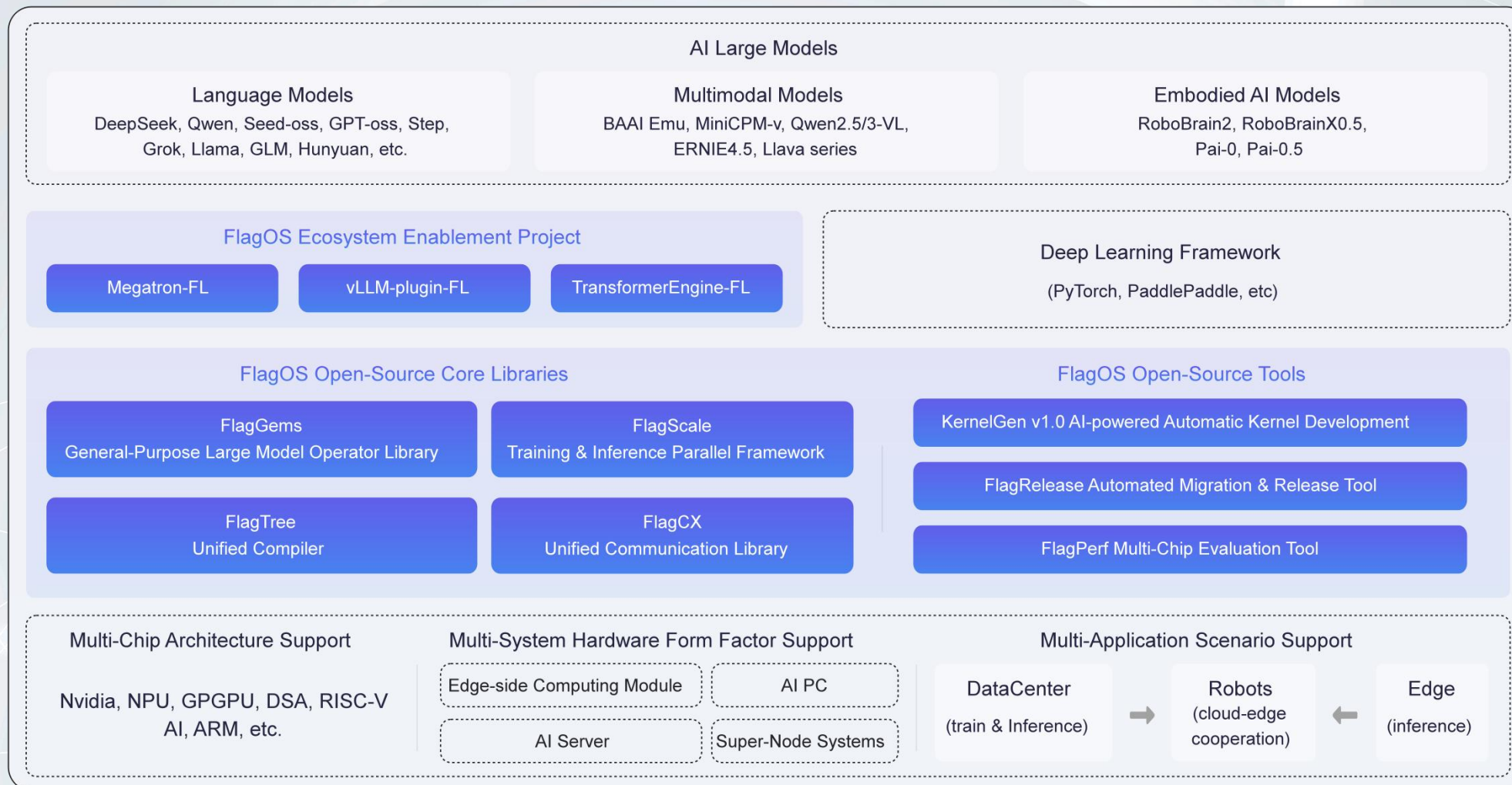
Use candidates supported by the probes; key = [M, N, K]

4 Confirm the winner

Compare exact-config fixed vs decorated autotune across shapes

A larger tile can improve reuse—or reduce parallelism and raise resource pressure.

The Solution: FlagOS, a Unified AI Software Stack





BAAI
智源研究院



北京大学
PEKING UNIVERSITY

Thanks for your time!